

Task 01 — Gameplay page and player actions

Owner: Badr. Branch: `member-badr`. Reviewer: Majed. Project deadline: 3 October 2026, 23:59 Asia/Damascus.

Context

Build the gameplay page: opening story, current stage, clues, answer form, feedback, and hint controls. Reuse useful parts of your existing interface and adapt them to the agreed structure.

Read:

- `docs/phase-1-agreement/styling-contract.pdf`: required palette, classes, component paths, and control states.
- `docs/phase-1-agreement/apis-handoff.pdf`: four APIs and backend-owned progress.
- `docs/phase-1-agreement/puzzles-reference.pdf`: original puzzle content.
- `docs/team/requirements.md`: project constraints.
- [Teacher client reference](#): function components, typed props, local state, API helpers, plain CSS, and routing syntax.

Eyad implements all backend endpoints. Majed provides shared API helpers/types and connects the frontend to the backend. Yazan owns selection, How to Play, shared layout/messages, and global CSS foundations. Alaa owns the result page and full-journey verification.

Goal

A player can open any puzzle, read its current stage, submit an answer, use allowed hints, and reach the result page using real server responses.

What to build

Helper content tree

Create or adapt the gameplay files below. Keep the shared dependency files with their owners. Use these component, function, prop, and state names; other temporary variables can use clear names.

```
client/
├── css/
│   └── index.css                shared with Yazan; add gameplay rules
├── only
│   ├── puzzle-panel, puzzle-story, stage-heading, clue-list,
│   │   │   clue-card, answer-form, answer-input, answer-feedback,
│   │   │   hint-panel; success/error/disabled states
│   └── src/
│       ├── pages/
│       │   ├── PuzzlePage.tsx    Badr
│       │   │   ├── PuzzlePage()
│       │   │   │   └── puzzleId    validated numeric route ID
```

```

├── components/
│   ├── PuzzleSection.tsx      Badr
│   │   ├── PuzzleSectionProps { puzzleId }
│   │   ├── PuzzleSection({ puzzleId })
│   │   ├── puzzle              PuzzleDetail | null
│   │   ├── answer              string
│   │   ├── feedback            { correct, message } | null
│   │   ├── loading             boolean
│   │   ├── pendingAction       "answer" | "hint" | null
│   │   ├── error               string | null; detail-load error
│   │   ├── actionError         string | null; answer/hint error
│   │   ├── loadPuzzle()
│   │   ├── handleAnswerChange(value)
│   │   ├── handleSubmitAnswer(event)
│   │   └── handleRequestHint()
│   ├── StageHeading.tsx      Badr
│   │   ├── StageHeadingProps { stage, progress }
│   │   └── StageHeading({ stage, progress })
│   ├── ClueList.tsx          Badr
│   │   ├── ClueListProps { clues }
│   │   └── ClueList({ clues })
│   ├── ClueCard.tsx          Badr
│   │   ├── ClueCardProps { text, number }
│   │   └── ClueCard({ text, number })
│   ├── AnswerForm.tsx        Badr
│   │   ├── AnswerFormProps { answer, pending, disabled,
onAnswerChange, onSubmit }
│   │   └── AnswerForm({ answer, pending, disabled, onAnswerChange,
onSubmit })
│   ├── AnswerFeedback.tsx    Badr
│   │   ├── AnswerFeedbackProps { correct, message }
│   │   └── AnswerFeedback({ correct, message })
│   ├── HintPanel.tsx         Badr
│   │   ├── HintPanelProps { hints, hintsRemaining, pending, disabled,
onRequestHint }
│   │   └── HintPanel({ hints, hintsRemaining, pending, disabled,
onRequestHint })
│   ├── LoadingMessage.tsx     Yazan; reuse
│   ├── ErrorMessage.tsx      Yazan; reuse
│   ├── api.ts                 Majed; import named async helpers
│   │   ├── getPuzzle(puzzleId)
│   │   ├── submitAnswer(puzzleId, stageId, answer)
│   │   └── requestStageHint(puzzleId, stageId)
│   ├── types.ts               Majed; import shared types
│   │   ├── PuzzleDetail, PuzzleStage, PuzzleProgress, RevealedHint,
SubmitAnswerResponse, StageHintResponse
│   ├── App.tsx                Yazan; shared routes/layout
│   │   └── /puzzles/:puzzleId → PuzzlePage
│   ├── main.tsx               Yazan; shared setup/CSS import
│   └── pages/ResultPage.tsx    Alaa; /puzzles/:puzzleId/result

```

- `PuzzlePage` reads `puzzleId` with `useParams()`. Reject invalid/non-positive/non-integer IDs with a readable message and a return link. Render `PuzzleSection` keyed by the valid puzzle ID so changing puzzles resets local screen state.
- `PuzzleSection` owns loading, input, feedback, and pending state. Load detail when the puzzle ID changes. Do not display the previous puzzle while loading another. Ignore responses from an old/unmounted page; use effect cleanup rather than a custom request framework.
- Show puzzle title, original opening story, current stage title, question, stage number, and clues. One layout handles all three stages and all three puzzles.
- Render clue strings without inventing evidence or changing wording. Use stable clue order and list keys. Display components receive props; they do not fetch.
- Use a labeled controlled answer input and a real form. Enter submits. Disable empty submissions and show pending button text. Prevent both answer and hint requests while either action is pending.
- Wrong answers show the server message and keep the stage and entered answer. Network/HTTP failures keep the input and allow retry. Clear the answer after a correct submission advances the stage.
- Show already revealed hints from the current stage. Keep the next-hint button available while `hintsRemaining > 0`; disable it with clear text when exhausted. Update hints from the server response.
- After an action, replace `puzzle` with the response's `puzzle`. Do not calculate stage advancement or hint usage locally. Avoid a background reload that could overwrite a newer action response.
- When `puzzle.progress.solved` is true, navigate to `/puzzles/:puzzleId/result`. Do not build a separate result implementation or display an invented ending.

API fields to use

Use Majed's named helpers; do not call `fetch` inside components or create a second helper module.

GET detail returns:

```
{
  id, title, summary, story,
  progress: { currentStageId, completedStages, totalStages, solved },
  currentStage: {
    id, title, question, clues,
    revealedHints: [{ hintId, hint }],
    hintsRemaining
  }
}
```

Puzzle/stage IDs are numbers; clues are strings. A fresh stage has an empty `revealedHints` array. When solved, `currentStage` and `currentStageId` are null and `finalReveal` is the original ending string.

- POST `/api/submitAnswer/:puzzleId`, body `{ stageId, answer }`, returns `{ correct, message, puzzle }`. A wrong answer is a successful HTTP 200 with `correct: false`.

- PATCH `/api/requestStageHint/:puzzleId`, body `{ stageId }`, returns `{ hintId, hint, puzzle }`. Hint IDs are zero-based stage-local indices. Never send a chosen hint ID.
- HTTP errors use `{ message }`: `400` invalid input, `404` missing puzzle/stage, `409` stale stage/solved puzzle/exhausted hints. Show readable feedback. For `409`, refresh the current puzzle before enabling more actions; it may have advanced through another visitor. Keep input unless the refreshed stage changed.
- Backend progress is shared between visitors and resets when the server restarts. Reloading the page alone does not reset the game.

Teacher syntax and styling

- Use named function components with default exports, `interface ComponentNameProps`, destructured props, and shared type imports from `types.ts`.
- Use `useState`, `useEffect`, named `load.../handle...` functions, typed `FormEvent`, and async handlers with `try/catch/finally`.
- Keep the selected puzzle in the URL and screen/input state local. Remove `createAsyncThunk` from this gameplay flow. Do not replace it with another request library or add Redux/Context for this task.
- Use real `Link` navigation and `useNavigate` after completion. Keep the page wrapper small and pass data/callbacks into components.
- Follow `styling-contract.pdf`: dark green, cream text, muted yellow actions, system font, prescribed classes, and visible feedback/focus/disabled states.
- Add gameplay styles to `client/css/index.css`, grouped with comments. Use plain CSS, kebab-case classes, Flexbox, hover/focus/disabled selectors, and responsive media queries as in the teacher reference. No `App.css`, Tailwind, CSS Modules, or styled-components.
- Coordinate the CSS file with Yazan. Transfer useful gameplay styling from your existing `App.css` into the agreed stylesheet. Yazan owns the shared import in `main.tsx` as `../css/index.css` and removal of the old `App.css` import from the shared shell.
- Adapt useful existing `MysteryRoom` markup to the named components. Remove obsolete gameplay imports only after replacements work. Preserve other members' responsibilities and your commit history.

Rules

- Implement gameplay only. Do not redesign selection, How to Play, shared navigation, or the result page.
- Yazan owns shared routes/layout/CSS foundations; Majed owns API helpers/types/proxy; Eyad owns backend state. Coordinate required shared-file changes with those owners.
- Follow project prohibitions: no `createAsyncThunk`, RTK Query, React Query, validation libraries, databases, authentication/sessions, Redis, WebSockets, GraphQL, Docker, deployment, or additional architecture frameworks.
- Do not import backend answers into the client, compare answers locally, use `localStorage` as progress authority, or leave temporary preview data/fake success in the delivered flow.
- Do not add timers, scores, replay, statistics, or new story content. Keep normal scrolling for long clues and usable controls on narrow screens.
- Work on `member-badr`. Do not force-push, merge into main yourself, or commit private documents, `.env`, `node_modules`, or generated build output.

Test it

1. Start the real backend and frontend. Open each puzzle through navigation and directly through its URL. Confirm original story, current stage, question, and clues.
2. Submit an empty answer: no request. Submit a wrong answer: readable feedback, same stage, answer retained. Submit a correct answer: next stage and cleared input. Verify Enter and the button both work.
3. Request both hints: IDs 0 then 1, both remain visible, next-hint action stays available after the first. Exhaustion disables the button. Reload: previously revealed current-stage hints remain.
4. While an action is pending, verify both action buttons are disabled. Stop the backend during an action: show a useful error, preserve input, release controls, and allow retry.
5. Complete the last stage: navigate to Alaa's result route. Reopen the solved gameplay URL: return to the result route using server state.
6. Test malformed and unknown puzzle URLs, a failed detail request with Retry, and backend `400/404/409` responses. No blank screen or unhandled promise. Change puzzles during loading: no previous-puzzle content appears.
7. Test around 375px width and desktop width, long clues, keyboard navigation, visible focus, labeled input, and feedback in words as well as colors.
8. Run `npm run build` and `npm run lint` in `client/`. Record results and screenshots of story/stage, wrong answer, hints, and narrow-screen layout. Verify the final flow uses real API helpers, with no preview fixtures or prohibited async thunks.

Done when

- All three puzzles use one gameplay layout connected to real endpoints.
- Answers, hints, loading, errors, and completion navigation work as specified.
- Component names, CSS location, palette, and syntax match the contract and reference patterns.
- Build/lint pass; you can explain the component props, request flow, and backend-owned progress.
- Majed has reviewed and merged the work.

Working time

Estimate: 3–4 focused hours. First checkpoint: current-stage display with real GET data. Second checkpoint: answer/hint actions and responsive checks. Raise blockers after 20–30 minutes. Prepare display components while shared helpers or backend work is being completed.

Submitting

Commit meaningful steps and push to `member-badr`. Open a PR to `main` and request Majed's review. Include build/lint results, runtime checks, screenshots, and known limitations. Submit gameplay changes separately from unrelated existing branch work so Majed can review and integrate the assigned scope.